



BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

MEKATRONİK MÜHENDİSLİĞİ BÖLÜMÜ

Yapay Sinir Ağlarına Giriş Projesi

MKTS0307

İbrahim İdris İbrahim – 20332629131

Samet Ozan Topcu- 19332652029

CNN(Convolutional Neural Networks)

Soru 1:

Verilen veri setinin özelliklerini ve model geliştirme öncesi uyguladığınız veri hazırlama adımlarınızı kısaca açıklayınız.

- 1.1) **Veri standardizasyonu:** Veri setindeki farklı özelliklerin farklı ölçeklerde olması durumunda, verileri aynı ölçeklere getirmek için standartlaştırma işlemi yapılabilir. Bu, verilerin ölçeklendirilmesi, normalleştirilmesi veya dönüştürülmesi gibi işlemleri içerir.

```
layers.experimental.preprocessing.Rescaling(1./255, input_shape=(image_height,image_width,3))
```

özellikle de buna veri normalizasyon denir. Normalleştirme (Normalization): Normalleştirme, verilerin belirli bir dağılım veya formata dönüştürülmesini sağlar. Veriler genellikle 0 ile 1 arasında veya -1 ile 1 arasında olacak şekilde normalleştirilir. Normalleştirme işlemi genellikle Min-Max ölçeklendirme yöntemiyle yapılır.

- 1.2) **Veri bölme:** Veri setinin eğitim ve test veri setlerine ayrılması gerekebilir. Bu, modelin eğitim veri seti üzerinde eğitilmesi ve test veri seti üzerinde doğrulama yapılması için verinin bölünmesini içerir.

```
train_ds=tf.keras.preprocessing.image_dataset_from_directory(data_dir, validation_split=0.2, subset = 'training', seed=123,
```

```
image_size=(image_height,image_width), batch_size=batch_size)
```

- 1.3) **Veri optimizasyonu:** TensorFlow veri setlerini daha verimli hale getirmek için kullanılan bazı optimizasyon tekniklerini içerir.

AUTOTUNE sabiti, TensorFlow veri setlerinin performansını artırmak için kullanılan bir parametredir. Bu parametre, veri setinin işleme hızını dinamik olarak ayarlamak için kullanılır. Bu sayede, veri setinin işlenmesi sırasında gereksiz bekleme süreleri azaltılarak işlem hızı artırılabilir.

cache() metodu, veri setinin belleğe alınmasını sağlar. Bu sayede, veri setinin tekrar tekrar diskten okunması yerine bellekten okunarak işlem hızı artırılabilir.

shuffle(1000) metodu, veri setinin elemanlarını karıştırarak eğitim sırasında modelin daha iyi öğrenmesini sağlar.

prefetch(buffer_size=AUTOTUNE) metodu, veri setinin önceden yüklenmesini sağlar. Bu sayede, model eğitilirken veri setinin diskten okunması ile modelin eğitim süreci arasında bekleme süreleri azaltılarak işlem hızı artırılabilir.

Bu kod parçacığı, eğitim ve doğrulama veri setlerinin performansını artırmak için yukarıda belirtilen teknikleri kullanarak veri setlerini optimize etmektedir.

```
AUTOTUNE = tf.data.experimental.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)

val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

soru 2 :

a) Basit bir mimariye sahip derin olmayan sığ bir ağ modeli oluşturun (örneğin, yalnızca bir gizli katman kullanarak, içinde yalnızca birkaç nöron/birim vb. kullanarak). Bu model, ilk seçtiğiniz yapay sinir ağ türündeki ilk versiyonunuz olacaktır. Sonra, oluşturduğunuz bu modelinizi eğitin.

2.1) Modelin açıklaması

layers.experimental.preprocessing.Rescaling(1./255, input_shape=(image_height, image_width, 3)): Bu katman, giriş görüntülerinin piksel değerlerini 0 ile 1 arasında ölçeklemek için kullanılır. 1./255 parametresi, piksel değerlerini 0 ile 255 arasından 0 ile 1 arasına dönüştürmek için kullanılan ölçekleme faktörüdür. **input_shape=(image_height, image_width, 3)** parametresi ise giriş görüntülerinin boyutunu ve kanal sayısını belirtir.

layers.Conv2D(16, 3, padding='same', activation='relu'): Bu katman, 16 adet 3x3 boyutunda filtre ile evrişim işlemi yapar. **padding='same'** parametresi, evrişim işlemi sonucunda çıktı boyutunun giriş boyutuna aynı olmasını sağlar. **activation='relu'** parametresi ise ReLU aktivasyon fonksiyonunu kullanarak çıkışı hesaplar.

layers.MaxPooling2D(): Bu katman, maksimum havuzlama işlemi uygular ve evrişim katmanının çıkışını küçültür. Varsayılan olarak 2x2 boyutunda bir filtre kullanır.

layers.Flatten(): Bu katman, evrişim ve havuzlama katmanlarının çıkışını düzleştirir ve tek boyutlu bir vektöre dönüştürür.

layers.Dense(128, activation='relu'): Bu katman, 128 nöronlu tam bağlı bir katmandır ve ReLU aktivasyon fonksiyonunu kullanır.

layers.Dense(num_classes): Bu katman, çıkış katmanıdır ve verilen **num_classes** sayısına göre nöronları içerir. Çok sınıflı bir sınıflandırma problemi için kullanılır.

2.2) Modelin özeti:

Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling_3 (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d (MaxPooling2D)	(None, 90, 90, 16)	0
flatten (Flatten)	(None, 129600)	0
dense (Dense)	(None, 128)	16588928
dense_1 (Dense)	(None, 2)	258
Total params: 16589634 (63.28 MB)		
Trainable params: 16589634 (63.28 MB)		
Non-trainable params: 0 (0.00 Byte)		

2.3 Model derleme kısmında ise aşağıdaki parametreler kullanılmıştır:

optimizer='adam': Modelin optimize edici olarak Adam optimizasyon algoritmasını kullanmasını belirtir.

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True): Modelin kayıp fonksiyonu olarak seyrek kategorik çapraz entropiyi kullanmasını belirtir. from_logits=True parametresi, modelin çıkışlarının logit değerler olduğunu belirtir.

metrics=['accuracy']: Modelin eğitim ve değerlendirme sırasında doğruluk metriğini kullanmasını belirtir. Bu metrik, modelin doğruluk performansını ölçer.

2.4 Model performansının değerlendirilmesi

veri setin görüntülemesi:

```
for image_batch, labels_batch in train_ds:  
  
    print(image_batch.shape)  
  
    print(labels_batch.shape)  
  
    break
```

```
(32, 64, 64, 3)
```

```
(32,)
```

Burada 32 batch boyutunu göstermektedir. 180 x 180 x 3 fotoğraflar bulunmaktadır. Sondaki 3 RGB renkleri göstermektedir.

Sonuçlar:

Epoch 1/5

211/211 [=====] - 237s 1s/step - loss: 0.3465 - accuracy: 0.8503 - val_loss: 0.3591 - val_accuracy: 0.8447

Epoch 2/5

211/211 [=====] - 233s 1s/step - loss: 0.3297 - accuracy: 0.8602 - val_loss: 0.3810 - val_accuracy: 0.8458

Epoch 3/5

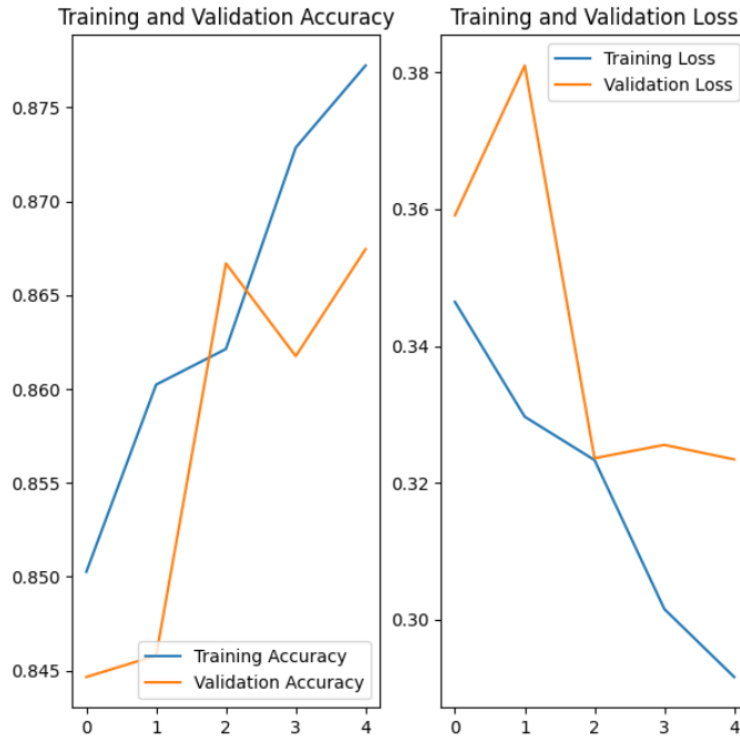
211/211 [=====] - 233s 1s/step - loss: 0.3233 - accuracy: 0.8621 - val_loss: 0.3236 - val_accuracy: 0.8667

Epoch 4/5

211/211 [=====] - 237s 1s/step - loss: 0.3016 - accuracy: 0.8729 - val_loss: 0.3256 - val_accuracy: 0.8618

Epoch 5/5

211/211 [=====] - 240s 1s/step - loss: 0.2916 - accuracy: 0.8772 - val_loss: 0.3234 - val_accuracy: 0.8675



Grafik 1: Epok değerine göre kayıp ve doğruluk grafikleri

Görselden de anlaşılacağı üzere, modelimiz öğrenme sürecinde beklenen performansı gösterememiş. Eğitim veri setinde ve doğrulama veri setindeki sonuçlar arasında büyük bir fark gözlemlenmiş, yani model eğitim sırasında yeterince öğrenememiş.

Sonuç olarak, modelimizi daha iyi eğitmek ve genelleme performansını artırmak için sıkı bir şekilde takip etmek ve uygun iyileştirmeleri yapmak önemlidir. Bu sayede daha dengeli ve etkili bir model elde edebiliriz.

Soru 3

Bir önceki soruda (soru 2) oluşturup eğittiğiniz ağınızı optimize edin:

Bir önceki model neredeyse hiçbir şey öğrenmemiş.

3.1) epoch sayısını artırmak:

veri sayısı fazla olduğu için epoch sayısını 5'ten 10'a çıkaralım.

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
rescaling (Rescaling)	(None, 64, 64, 3)	0
conv2d (Conv2D)	(None, 64, 64, 16)	448

max_pooling2d (MaxPooling2 (None, 90, 90, 16) 0

D)

flatten (Flatten) (None, 129600) 0

dense (Dense) (None, 128) 16588928

dense_1 (Dense) (None, 2) 258

=====

Total params: 16589634 (63.28 MB)

Trainable params: 16589634 (63.28 MB)

Non-trainable params: 0 (0.00 Byte)

Sonuçlar

Epoch 1/10

211/211 [=====] - 374s 2s/step - loss: 0.7742 - accuracy: 0.7617 - val_loss: 0.4098 - val_accuracy: 0.7896

Epoch 2/10

211/211 [=====] - 249s 1s/step - loss: 0.3650 - accuracy: 0.8417 - val_loss: 0.3204 - val_accuracy: 0.8652

Epoch 3/10

211/211 [=====] - 245s 1s/step - loss: 0.3278 - accuracy: 0.8589 - val_loss: 0.3059 - val_accuracy: 0.8769

Epoch 4/10

211/211 [=====] - 249s 1s/step - loss: 0.3109 - accuracy: 0.8711 - val_loss: 0.3042 - val_accuracy: 0.8781

Epoch 5/10

211/211 [=====] - 243s 1s/step - loss: 0.2891 - accuracy: 0.8806 - val_loss: 0.2897 - val_accuracy: 0.8811

Epoch 6/10

211/211 [=====] - 241s 1s/step - loss: 0.2703 - accuracy: 0.8881 - val_loss: 0.2735 - val_accuracy: 0.8868

Epoch 7/10

211/211 [=====] - 241s 1s/step - loss: 0.2516 - accuracy: 0.8960 - val_loss: 0.3408 - val_accuracy: 0.8538

Epoch 8/10

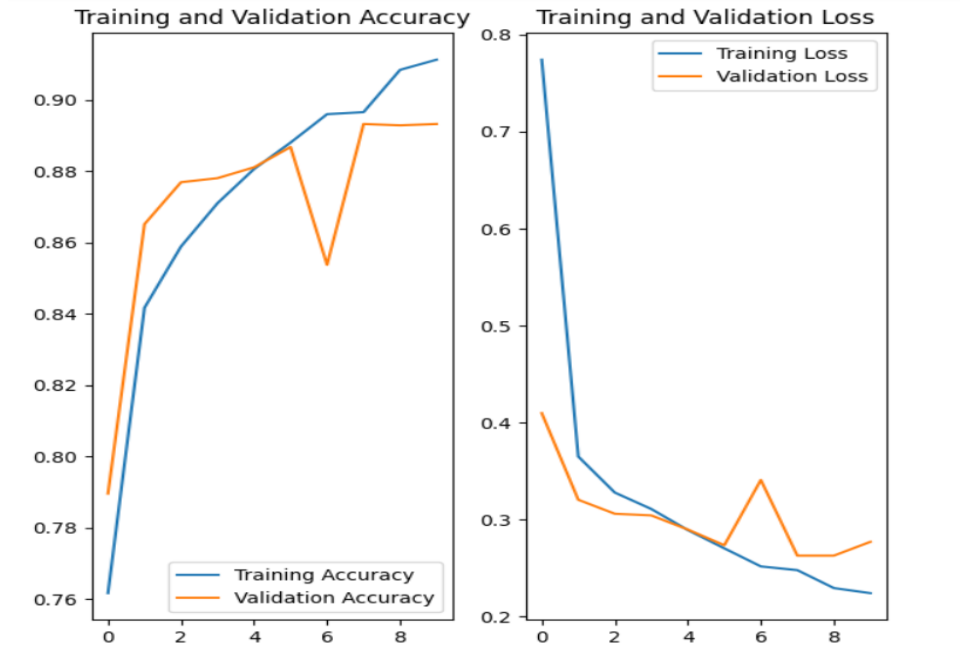
211/211 [=====] - 242s 1s/step - loss: 0.2478 - accuracy: 0.8966 - val_loss: 0.2628 - val_accuracy: 0.8933

Epoch 9/10

211/211 [=====] - 240s 1s/step - loss: 0.2293 - accuracy: 0.9085 - val_loss: 0.2627 - val_accuracy: 0.8929

Epoch 10/10

211/211 [=====] - 242s 1s/step - loss: 0.2241 - accuracy: 0.9113 - val_loss: 0.2770 - val_accuracy: 0.8933



Grafik 2: Epok değerine göre kayıp ve doğruluk grafikleri

Epoch sayısının artırılması modelimizin performansını olumlu yönde etkiliyor gibi görünmektedir. Ancak, yaklaşık 5. epoch'tan sonra eğitim ve doğrulama kayıpları arasındaki farkın giderek arttığı gözlemlenmiştir. Bu durum, modelin aşırı öğrenmeye (overfitting) başladığını göstermektedir.

3.2) ara katmanları ekleme :

layers.Conv2D'deki ilk parametre, evrişim katmanının öğreneceği filtre sayısını (aynı zamanda çekirdekler veya kanallar olarak da bilinir) temsil eder. Verilen kod parçasığında, 16, 32 ve 64

sayıları üç Conv2D katmanı için filtre sayısını temsil eder. Her filtre, girdi verilerinde belirli bir deseni veya özelliği tespit etmeyi öğrenir.

```
layers.Conv2D(32, 3, padding='same', activation='relu'),  
layers.MaxPooling2D(),  
layers.Conv2D(64, 3, padding='same', activation='relu'),  
layers.MaxPooling2D(),
```

Sonuçlar :

Epoch 1/10

211/211 [=====] - 346s 2s/step - loss: 0.0792 - accuracy: 0.9691 - val_loss: 0.3051 - val_accuracy: 0.9149

Epoch 2/10

211/211 [=====] - 345s 2s/step - loss: 0.0587 - accuracy: 0.9788 - val_loss: 0.3348 - val_accuracy: 0.9077

Epoch 3/10

211/211 [=====] - 346s 2s/step - loss: 0.0326 - accuracy: 0.9889 - val_loss: 0.3721 - val_accuracy: 0.9126

Epoch 4/10

211/211 [=====] - 343s 2s/step - loss: 0.0339 - accuracy: 0.9877 - val_loss: 0.4392 - val_accuracy: 0.9115

Epoch 5/10

211/211 [=====] - 343s 2s/step - loss: 0.0125 - accuracy: 0.9971 - val_loss: 0.4955 - val_accuracy: 0.9161

Epoch 6/10

211/211 [=====] - 344s 2s/step - loss: 0.0150 - accuracy: 0.9951 - val_loss: 0.5047 - val_accuracy: 0.9077

Epoch 7/10

211/211 [=====] - 345s 2s/step - loss: 0.0153 - accuracy: 0.9956 - val_loss: 0.5248 - val_accuracy: 0.9092

Epoch 8/10

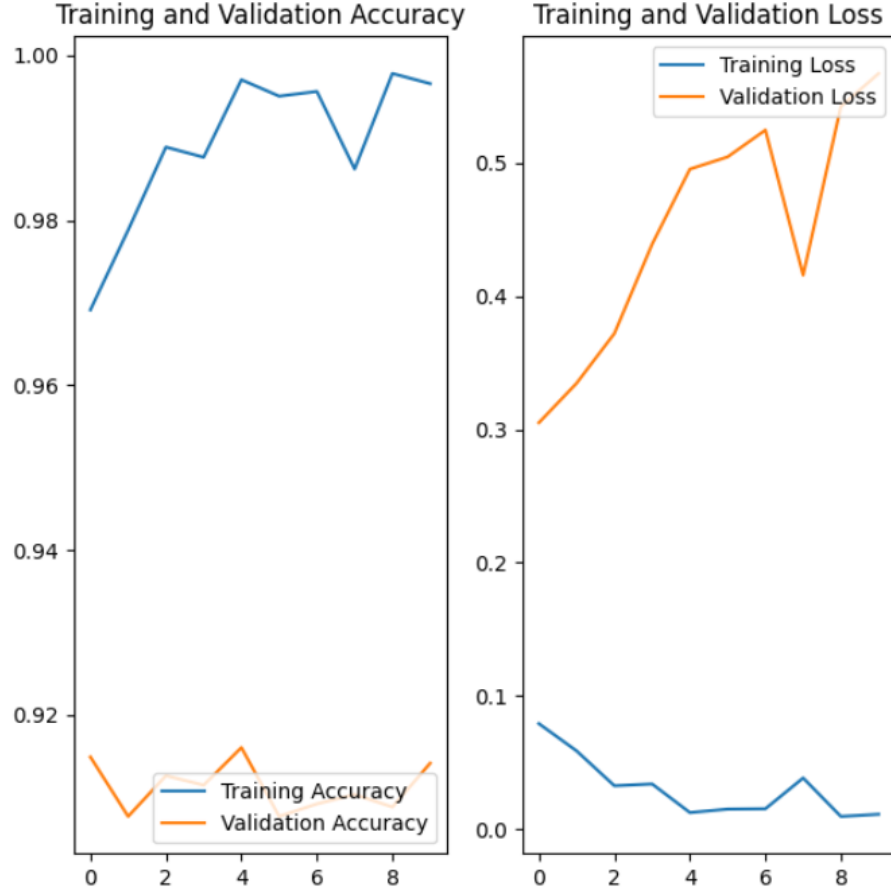
211/211 [=====] - 362s 2s/step - loss: 0.0384 - accuracy: 0.9862 - val_loss: 0.4159 - val_accuracy: 0.9104

Epoch 9/10

211/211 [=====] - 347s 2s/step - loss: 0.0096 - accuracy: 0.9978 - val_loss: 0.5425 - val_accuracy: 0.9088

Epoch 10/10

211/211 [=====] - 345s 2s/step - loss: 0.0112 - accuracy: 0.9966 - val_loss: 0.5673 - val_accuracy: 0.9142



Grafik 3: Epok değerine göre kayıp ve doğruluk grafikleri

Ara katmanların eklenmesinin, modelin doğrudan aşırı öğrenmeye (overfitting) yol açtığı görülmektedir.

3.3) Drop out :

Dropout, yapay sinir ağlarında aşırı uyum (overfitting) sorununu azaltmak için kullanılan bir regularizasyon tekniğidir. Dropout, eğitim sırasında rastgele seçilen bir yüzde oranındaki nöronları devre dışı bırakarak (deaktif ederek) ağın genel yapısını zayıflatır.

Dropout, ağın aşırı uyuma (overfitting) eğilimini azaltır ve ağın daha iyi genelleme yapmasını sağlar. Bu sayede eğitim verilerine aşırı derecede uyum sağlayarak test verilerinde başarısız olma riski azalır.

layers.Dropout(0.3)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
=====		
rescaling_2 (Rescaling)	(None, 64, 64, 3)	0
conv2d_4 (Conv2D)	(None, 64, 64, 16)	448
max_pooling2d_4 (MaxPoolin g2D)	(None, 90, 90, 16)	0
conv2d_5 (Conv2D)	(None, 90, 90, 32)	4640
max_pooling2d_5 (MaxPoolin g2D)	(None, 45, 45, 32)	0
conv2d_6 (Conv2D)	(None, 45, 45, 64)	18496
max_pooling2d_6 (MaxPoolin g2D)	(None, 22, 22, 64)	0
dropout (Dropout)	(None, 22, 22, 64)	0
flatten_2 (Flatten)	(None, 30976)	0
dense_4 (Dense)	(None, 128)	3965056
dense_5 (Dense)	(None, 2)	258
=====		

Total params: 3988898 (15.22 MB)

Trainable params: 3988898 (15.22 MB)

Non-trainable params: 0 (0.00 Byte)

Sonuçlar :

Epoch 1/10

211/211 [=====] - 367s 2s/step - loss: 0.4901 - accuracy: 0.7587 - val_loss: 0.3808 - val_accuracy: 0.8564

Epoch 2/10

211/211 [=====] - 354s 2s/step - loss: 0.3312 - accuracy: 0.8582 - val_loss: 0.2838 - val_accuracy: 0.8834

Epoch 3/10

211/211 [=====] - 354s 2s/step - loss: 0.2823 - accuracy: 0.8817 - val_loss: 0.2558 - val_accuracy: 0.8990

Epoch 4/10

211/211 [=====] - 354s 2s/step - loss: 0.2542 - accuracy: 0.8992 - val_loss: 0.2456 - val_accuracy: 0.9024

Epoch 5/10

211/211 [=====] - 352s 2s/step - loss: 0.2357 - accuracy: 0.9050 - val_loss: 0.2575 - val_accuracy: 0.8978

Epoch 6/10

211/211 [=====] - 367s 2s/step - loss: 0.2203 - accuracy: 0.9102 - val_loss: 0.2291 - val_accuracy: 0.9070

Epoch 7/10

211/211 [=====] - 349s 2s/step - loss: 0.1977 - accuracy: 0.9220 - val_loss: 0.2204 - val_accuracy: 0.9142

Epoch 8/10

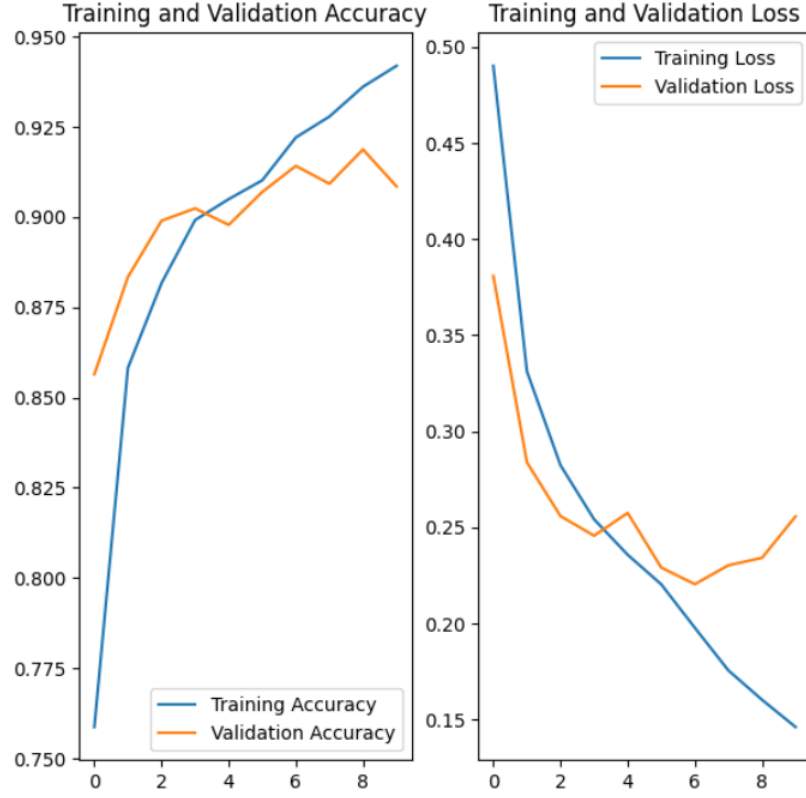
211/211 [=====] - 366s 2s/step - loss: 0.1755 - accuracy: 0.9278 - val_loss: 0.2303 - val_accuracy: 0.9092

Epoch 9/10

211/211 [=====] - 346s 2s/step - loss: 0.1603 - accuracy: 0.9361 - val_loss: 0.2341 - val_accuracy: 0.9187

Epoch 10/10

211/211 [=====] - 366s 2s/step - loss: 0.1461 - accuracy: 0.9419 - val_loss: 0.2557 - val_accuracy: 0.9085



Grafik 4: Epok değerine göre kayıp ve doğruluk grafikleri

Dropout uygulanmasının modelin performansını belirgin bir şekilde etkilediği açıkça görülmektedir. Ancak, 6. epoch'tan sonra modelin aşırı öğrenmeye (overfitting) başladığı anlaşılmaktadır.

3.4) modeli basitleştirmek

```
layers.Flatten(),
layers.Dense(100, activation='relu'),
layers.Dense(num_classes)]
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
=====		
rescaling_3 (Rescaling)	(None, 64, 64, 3)	0

conv2d_7 (Conv2D)	(None, 64, 64, 16)	448
max_pooling2d_7 (MaxPoolin	(None, 90, 90, 16)	0
g2D)		
conv2d_8 (Conv2D)	(None, 90, 90, 32)	4640
max_pooling2d_8 (MaxPoolin	(None, 45, 45, 32)	0
g2D)		
conv2d_9 (Conv2D)	(None, 45, 45, 32)	9248
max_pooling2d_9 (MaxPoolin	(None, 22, 22, 32)	0
g2D)		
dropout_1 (Dropout)	(None, 22, 22, 32)	0
flatten_3 (Flatten)	(None, 15488)	0
dense_6 (Dense)	(None, 128)	1982592
dense_7 (Dense)	(None, 2)	258

=====

Total params: 1997186 (7.62 MB)

Trainable params: 1997186 (7.62 MB)

Non-trainable params: 0 (0.00 Byte)

Sonuçlar:

Epoch 1/10

211/211 [=====] - 362s 2s/step - loss: 0.4133 - accuracy: 0.8140 - val_loss: 0.3294 - val_accuracy: 0.8675

Epoch 2/10

211/211 [=====] - 360s 2s/step - loss: 0.2937 - accuracy: 0.8791 - val_loss: 0.2815 - val_accuracy: 0.8857

Epoch 3/10

211/211 [=====] - 342s 2s/step - loss: 0.2624 - accuracy: 0.8941 - val_loss: 0.2584 - val_accuracy: 0.9020

Epoch 4/10

211/211 [=====] - 341s 2s/step - loss: 0.2474 - accuracy:
0.9031 - val_loss: 0.2437 - val_accuracy: 0.9066

Epoch 5/10

211/211 [=====] - 345s 2s/step - loss: 0.2259 - accuracy:
0.9105 - val_loss: 0.2375 - val_accuracy: 0.9020

Epoch 6/10

211/211 [=====] - 343s 2s/step - loss: 0.2023 - accuracy:
0.9189 - val_loss: 0.2440 - val_accuracy: 0.8997

Epoch 7/10

211/211 [=====] - 344s 2s/step - loss: 0.1896 - accuracy:
0.9240 - val_loss: 0.2389 - val_accuracy: 0.9054

Epoch 8/10

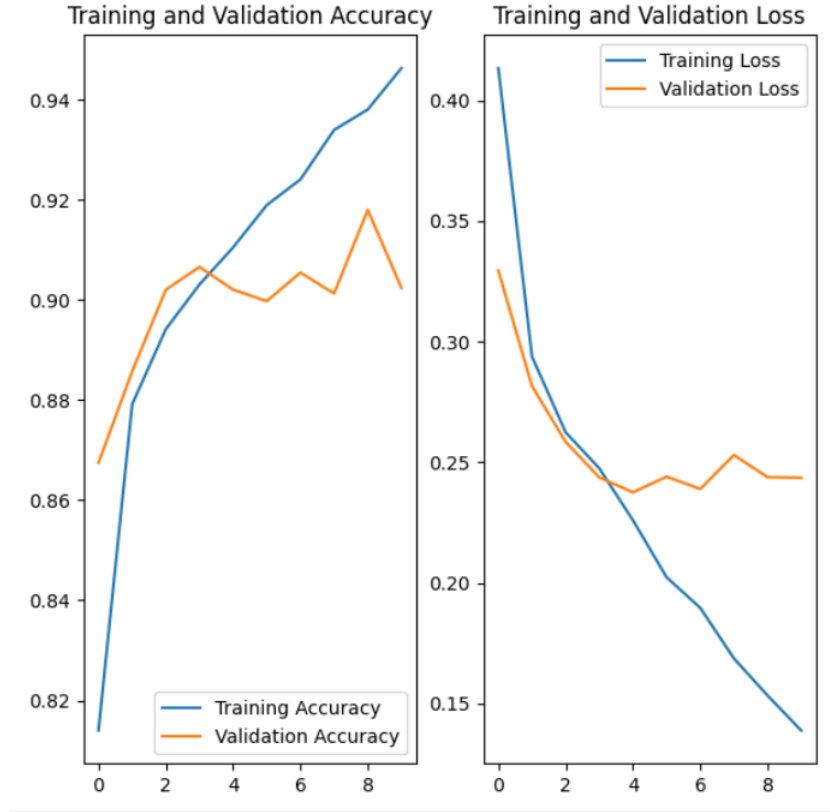
211/211 [=====] - 344s 2s/step - loss: 0.1687 - accuracy:
0.9339 - val_loss: 0.2530 - val_accuracy: 0.9013

Epoch 9/10

211/211 [=====] - 344s 2s/step - loss: 0.1533 - accuracy:
0.9380 - val_loss: 0.2438 - val_accuracy: 0.9180

Epoch 10/10

211/211 [=====] - 344s 2s/step - loss: 0.1388 - accuracy:
0.9463 - val_loss: 0.2436 - val_accuracy: 0.9024



Grafik 5: Epok değerine göre kayıp ve doğruluk grafikleri

Önceki aşırı öğrenme problemi çözmek amacıyla model basitleştirilerek yeniden eğitildi. Ancak, bu sefer modelin çok daha erken aşırı öğrenmeye başladığı gözlemlenmektedir.

3.5) kernel boyutu değiştirilmesi:

```
layers.Conv2D(32, 3, padding='same', activation='relu'),  
layers.MaxPooling2D(),  
layers.Conv2D(64, 2, padding='same', activation='relu'),
```

Sonuçlar :

Epoch 1/10

211/211 [=====] - 324s 2s/step - loss: 0.4736 - accuracy: 0.7763 - val_loss: 0.3519 - val_accuracy: 0.8538

Epoch 2/10

211/211 [=====] - 321s 2s/step - loss: 0.3124 - accuracy: 0.8676 - val_loss: 0.2915 - val_accuracy: 0.8769

Epoch 3/10

211/211 [=====] - 320s 2s/step - loss: 0.2798 - accuracy: 0.8830 - val_loss: 0.2642 - val_accuracy: 0.8925

Epoch 4/10

211/211 [=====] - 323s 2s/step - loss: 0.2546 - accuracy: 0.8969 - val_loss: 0.2468 - val_accuracy: 0.8994

Epoch 5/10

211/211 [=====] - 338s 2s/step - loss: 0.2355 - accuracy: 0.9047 - val_loss: 0.2492 - val_accuracy: 0.9032

Epoch 6/10

211/211 [=====] - 316s 1s/step - loss: 0.2226 - accuracy: 0.9089 - val_loss: 0.2365 - val_accuracy: 0.9070

Epoch 7/10

211/211 [=====] - 319s 2s/step - loss: 0.1973 - accuracy: 0.9212 - val_loss: 0.2269 - val_accuracy: 0.9092

Epoch 8/10

211/211 [=====] - 320s 2s/step - loss: 0.1897 - accuracy: 0.9241 - val_loss: 0.2655 - val_accuracy: 0.8910

Epoch 9/10

211/211 [=====] - 318s 2s/step - loss: 0.1788 - accuracy: 0.9276 - val_loss: 0.2359 - val_accuracy: 0.9126

Epoch 10/10

211/211 [=====] - 335s 2s/step - loss: 0.1593 - accuracy: 0.9385 - val_loss: 0.2692 - val_accuracy: 0.9088



Grafik 6: Epok değerine göre kayıp ve doğruluk grafikleri

Kernel boyutun azaltılması sistemin iyi yönde etkilediği görülmektedir. CNN'de (Convolutional Neural Network) kernel boyutu, genellikle bir konvolüsyon katmanında kullanılan filtrelerin (kernellerin) genişlik ve yükseklik boyutlarını ifade eder. Örneğin, 3x3 bir kernel boyutu, her bir filtrenin 3 piksel genişliğinde ve 3 piksel yüksekliğinde olduğu anlamına gelir.

3.6) Learning rate değerin değiştirilmesi:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
```

Layer (type)	Output Shape	Param #
=====		
rescaling (Rescaling)	(None, 64, 64, 3)	0
conv2d (Conv2D)	(None, 64, 64, 16)	448
max_pooling2d (MaxPooling2D)	(None, 32, 32, 16)	0
conv2d_1 (Conv2D)	(None, 32, 32, 32)	4640

max_pooling2d_1 (MaxPooling (None, 16, 16, 32) 0

2D)

conv2d_2 (Conv2D) (None, 16, 16, 64) 8256

max_pooling2d_2 (MaxPooling (None, 8, 8, 64) 0

2D)

dropout (Dropout) (None, 8, 8, 64) 0

flatten (Flatten) (None, 4096) 0

dense (Dense) (None, 120) 491640

dense_1 (Dense) (None, 2) 242

=====

Total params: 505,226

Trainable params: 505,226

Non-trainable params: 0

Epoch 1/7

211/211 [=====] - 18s 71ms/step - loss: 0.3928 - accuracy: 0.8246 - val_loss: 0.2720 - val_accuracy: 0.8906

Epoch 2/7

211/211 [=====] - 15s 70ms/step - loss: 0.2695 - accuracy: 0.8955 - val_loss: 0.2420 - val_accuracy: 0.9039

Epoch 3/7

211/211 [=====] - 15s 70ms/step - loss: 0.2295 - accuracy: 0.9117 - val_loss: 0.2141 - val_accuracy: 0.9111

Epoch 4/7

211/211 [=====] - 15s 69ms/step - loss: 0.2164 - accuracy: 0.9171 - val_loss: 0.2072 - val_accuracy: 0.9195

Epoch 5/7

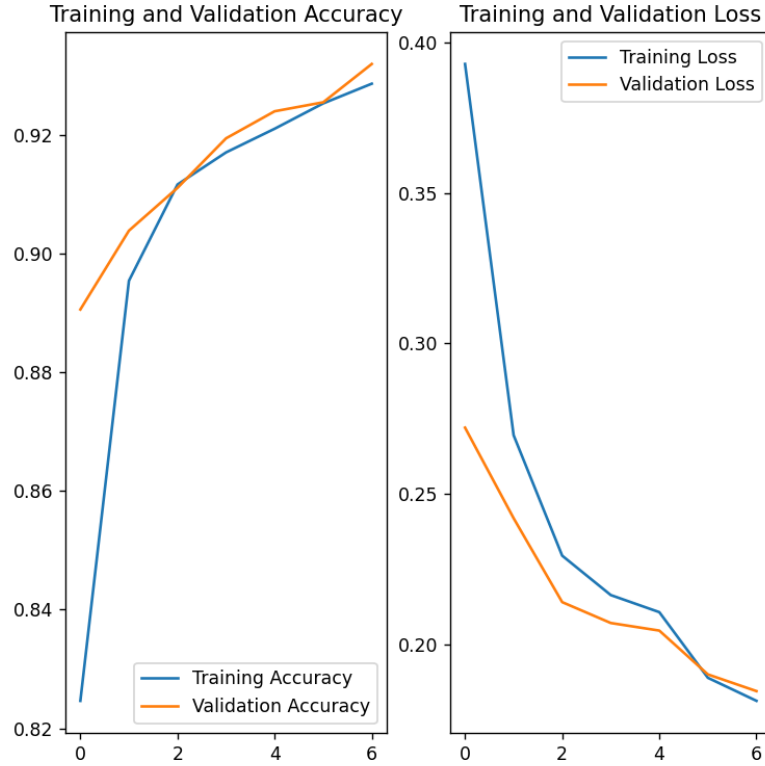
211/211 [=====] - 14s 68ms/step - loss: 0.2108 - accuracy: 0.9211 - val_loss: 0.2046 - val_accuracy: 0.9240

Epoch 6/7

211/211 [=====] - 14s 66ms/step - loss: 0.1889 - accuracy: 0.9254 - val_loss: 0.1901 - val_accuracy: 0.9256

Epoch 7/7

211/211 [=====] - 14s 66ms/step - loss: 0.1813 - accuracy: 0.9287 - val_loss: 0.1845 - val_accuracy: 0.9320



Grafik 7: Epok değerine göre kayıp ve doğruluk grafikleri

Kodun colab bağlantısı:

<https://colab.research.google.com/drive/1Pua-eDADlbLHgAnBeiZZg81vBIYSVgVM?authuser=0#scrollTo=vD0Y6uOQdRdC>

İleri Beslemeli Sinir Ağı (Fully Connected Feed Forward Neural Network)

Soru 1:

Verilen veri setinin özelliklerini ve model geliştirme öncesi uyguladığınız veri hazırlama adımlarınızı kısaca açıklayınız.

1.1 Veri Standardizasyonu: Veri setindeki farklı özelliklerin farklı ölçeklerde olması durumunda, verileri aynı ölçeklere getirmek için standartlaştırma işlemi yapılabilir. Bu, verilerin ölçeklendirilmesi, normalleştirilmesi veya dönüştürülmesi gibi işlemleri içerir.

```
normalization_layer = layers.experimental.preprocessing.Rescaling(1./255)

normalized_train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))

normalized_val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
```

1.2 Veri Bölme: Veri setinin eğitim ve test veri setlerine ayrılması gerekebilir. Bu, modelin eğitim veri seti üzerinde eğitilmesi ve test veri seti üzerinde doğrulama yapılması için verinin bölünmesini içerir.

```
val_ds = tf.keras.preprocessing.image_dataset_from_directory( data_dir,
validation_split=0.2, subset="validation", seed=123, image_size=(input_size,
seq_length),batch_size=batch_size)
```

1.3 Veri Optimizasyonu: TensorFlow veri setlerini daha verimli hale getirmek için kullanılan bazı optimizasyon tekniklerini içerir.

```
AUTOTUNE = tf.data.experimental.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)

val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Soru 2 :

a) Basit bir mimariye sahip derin olmayan sıg bir ağ modeli oluşturun (örneğin, yalnızca bir gizli katman kullanarak, içinde yalnızca birkaç nöron/birim vb. kullanarak). Bu model, ilk seçtiğiniz yapay sinir ağ türündeki ilk versiyonunuz olacaktır. Sonra, oluşturduğunuz bu modelinizi eğitin.

2.1 Modelin Açıklaması :

İlk olarak, veri kümesinin yüklenmesi için **tf.keras.preprocessing.image_dataset_from_directory** fonksiyonu kullanılmaktadır. Bu fonksiyon, belirtilen dizinden görüntüleri yükleyerek bir veri seti oluşturur. Veri kümesinin eğitim ve doğrulama alt kümelerine ayrılması amacıyla **validation_split** parametresi kullanılmaktadır.

Sonrasında, veri kümesinin normalleştirilmesi için **layers.experimental.preprocessing.Rescaling** katmanı kullanılmaktadır. Bu katman, görüntü piksellerinin 0 ile 1 arasında ölçeklendirilmesini sağlamaktadır.

Modelin oluşturulması sürecinde, **tf.keras.Sequential** kullanılarak ardışık bir model tanımlanmaktadır. Model, bir **Flatten** katmanı ile başlamakta ve ardından iki **Dense** katman (128 nöronlu ve softmax aktivasyonlu bir çıkış katmanı) eklenmektedir.

Model derlenirken, 'adam' optimizasyon algoritması ve 'SparseCategoricalCrossentropy' kayıp fonksiyonu kullanılmaktadır. Modelin değerlendirilmesinde ise 'accuracy' metriği kullanılmaktadır.

Son olarak, modelin eğitilmesi **model.fit** yöntemi ile gerçekleştirilmektedir. Eğitim sürecinde, **normalized_train_ds** ve **normalized_val_ds** veri setleri kullanılarak belirtilen epoch sayısında model eğitilmektedir.

Kodun son kısımlarında, eğitim sürecinde modelin performansını değerlendirmek ve gözlemek amacıyla modelin grafiği çizilmektedir

2.2 Model Özeti:

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 97200)	0
dense (Dense)	(None, 128)	1244172
dense_1 (Dense)	(None, 2)	258

Total params: 12,441,986

Trainable params: 12,441,986

Non-trainable params: 0

2.3 Model Derleme Kısımında İse Aşağıdaki Parametreler Kullanılmıştır:

1. **optimizer='adam':** Bu parametre, modelin hangi optimizier kullanılacağını belirler. Optimizier, modelin eğitim aşamasında parametrelerini güncellerken kullanılan algoritmayı temsil eder. 'adam', adaptif momentum tahminlerini kullanan bir türevidir ve genellikle görüntü işleme ve derin öğrenme modellerinde yaygın olarak tercih edilir.
2. **loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True):** Bu parametre, modelin eğitim sırasında kullanacağı kayıp fonksiyonunu belirler. SparseCategoricalCrossentropy, sınıflandırma problemleri için kullanılan bir kayıp fonksiyonudur. from_logits=True parametresi, modelin çıktılarının logitler olduğunu belirtir ve bu durumda kayıp fonksiyonu sağlanan logitler üzerinden hesaplanır.
3. **metrics=['accuracy']:** Bu parametre, modelin eğitim ve değerlendirme aşamalarında takip edilecek metrikleri belirler. 'accuracy' metriği, modelin doğruluk değerini takip etmesini sağlar ve sınıflandırma doğruluğunu hesaplar.

Sonuçlar:

330/330 [=====] - 18s 53ms/step - loss: 1.5024 - accuracy: 0.7591 - val_loss: 0.3594 - val_accuracy: 0.8454

Epoch 2/10

330/330 [=====] - 17s 50ms/step - loss: 0.4236 - accuracy: 0.8164 - val_loss: 0.5432 - val_accuracy: 0.7881

Epoch 3/10

330/330 [=====] - 16s 48ms/step - loss: 0.4017 - accuracy: 0.8294 - val_loss: 0.4093 - val_accuracy: 0.8283

Epoch 4/10

330/330 [=====] - 16s 47ms/step - loss: 0.4029 - accuracy: 0.8224 - val_loss: 0.5520 - val_accuracy: 0.7763

Epoch 5/10

330/330 [=====] - 16s 47ms/step - loss: 0.3652 - accuracy: 0.8431 - val_loss: 0.5207 - val_accuracy: 0.7824

Epoch 6/10

330/330 [=====] - 16s 49ms/step - loss: 0.3612 - accuracy: 0.8404 - val_loss: 0.3369 - val_accuracy: 0.8542

Epoch 7/10

330/330 [=====] - 16s 48ms/step - loss: 0.3547 - accuracy: 0.8471 - val_loss: 0.3359 - val_accuracy: 0.8583

Epoch 8/10

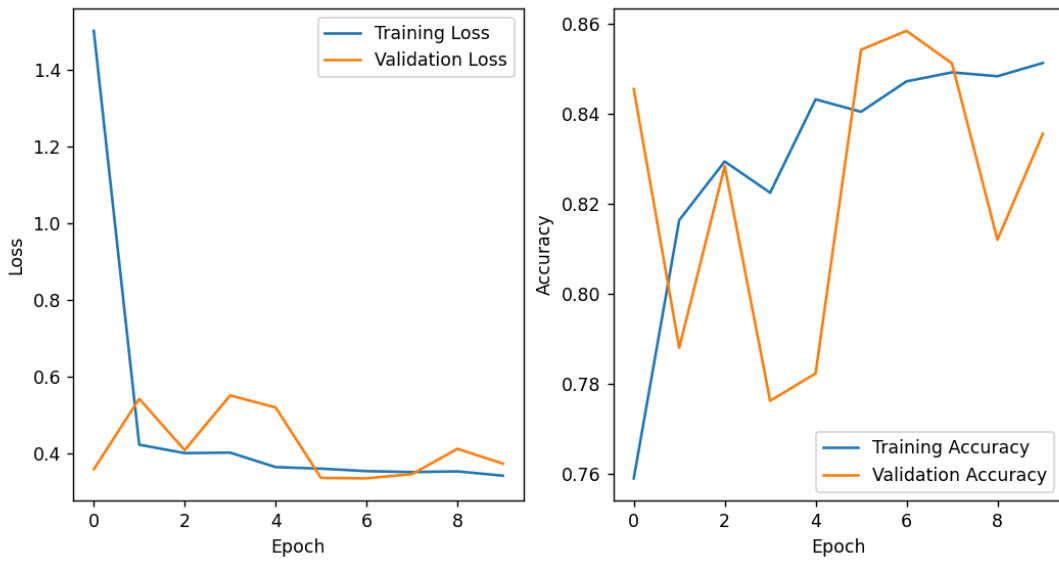
330/330 [=====] - 18s 54ms/step - loss: 0.3523 - accuracy: 0.8491 - val_loss: 0.3469 - val_accuracy: 0.8511

Epoch 9/10

330/330 [=====] - 17s 50ms/step - loss: 0.3541 - accuracy: 0.8483 - val_loss: 0.4128 - val_accuracy: 0.8120

Epoch 10/10

330/330 [=====] - 16s 50ms/step - loss: 0.3429 - accuracy: 0.8512 - val_loss: 0.3743 - val_accuracy: 0.8355



Grafik 8: Epok değerine göre kayıp ve doğruluk grafikleri

Yukarıda verilen şekilde modelin test veri setinde çok kötü performansa sahip olduğu görülmektedir. Çok büyük ve ani sıçramalar modelin az eğitildiğini göstermektedir.

Soru 3:

Bir önceki soruda (soru 2) oluşturup eğittiğiniz ağını optimize edin:

3.1 Learning Rate Parametresinin Eklenmesi:

Öğrenme hızı (learning rate), derin öğrenme modellerinin eğitim sürecinde kullanılan bir hiperparametredir ve modelin ağırlık güncellemelerinin büyüklüğünü belirler. Başka bir deyişle, her bir eğitim adımında model parametrelerinin ne kadar değiştirileceğini kontrol eder. Yüksek bir öğrenme hızı seçildiğinde, model daha hızlı öğrenir; ancak, bu durum ağırlıkların hızlı değişmesi nedeniyle modelin kararsız hale gelmesine veya ağırlıkların sapmasına yol açabilir. Düşük bir öğrenme hızı seçildiğinde ise model daha istikrarlı bir şekilde öğrenir; ancak, bu durumda eğitim süresi uzayabilir.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```


Sonuçlar:

Model: "sequential"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

=====

flatten (Flatten)	(None, 97200)	0
dense (Dense)	(None, 128)	12441728
dense_1 (Dense)	(None, 300)	38700
dense_2 (Dense)	(None, 2)	602

=====

Total params: 12,481,030

Trainable params: 12,481,030

Non-trainable params: 0

Epoch 5/10

330/330 [=====] - 16s 48ms/step - loss: 0.3988 - accuracy: 0.8212 - val_loss: 0.4338 - val_accuracy: 0.7144

Epoch 6/10

330/330 [=====] - 16s 48ms/step - loss: 0.4608 - accuracy: 0.7298 - val_loss: 0.5303 - val_accuracy: 0.7790

Epoch 7/10

330/330 [=====] - 16s 47ms/step - loss: 0.4087 - accuracy: 0.8319 - val_loss: 0.4341 - val_accuracy: 0.7915

Epoch 8/10

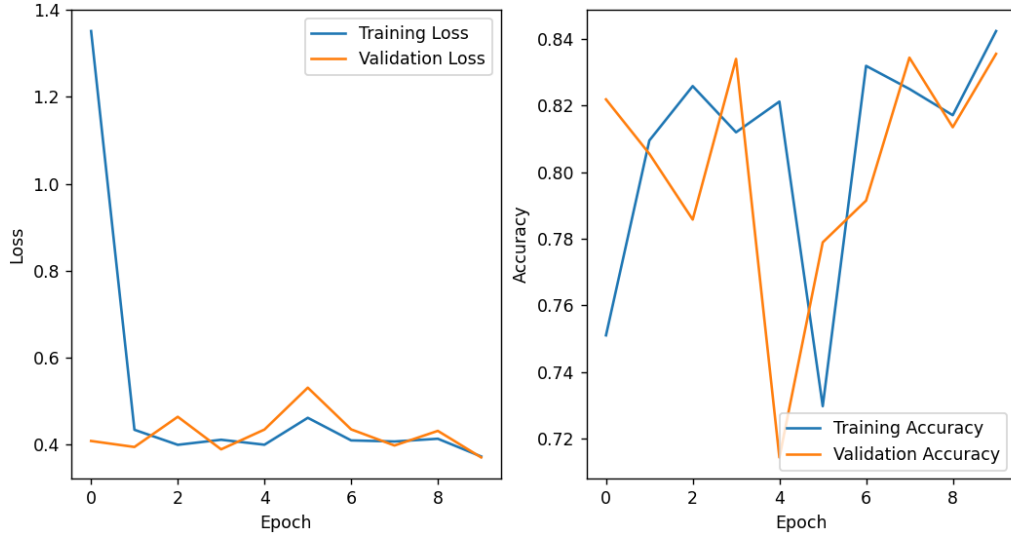
330/330 [=====] - 16s 48ms/step - loss: 0.4062 - accuracy: 0.8250 - val_loss: 0.3970 - val_accuracy: 0.8344

Epoch 9/10

330/330 [=====] - 16s 49ms/step - loss: 0.4125 - accuracy: 0.8171 - val_loss: 0.4307 - val_accuracy: 0.8135

Epoch 10/10

330/330 [=====] - 16s 49ms/step - loss: 0.3717 - accuracy: 0.8424 - val_loss: 0.3695 - val_accuracy: 0.8355



Grafik 9: Epok değerine göre kayıp ve doğruluk grafikleri

Burada performans artırmak için yapılan değişikliklere rağmen model çok daha kötü performans sergilemektedir. Başka optimizasyon yöntemleri kullanmak lazım.

3.2 Learning_Rate'i Azaltmak / Artırmak :

Grafikten anlaşılacağı üzere ani sıçramalar olmaktadır. Bunun nedeni ise öğrenme oranının çok yüksek olmasıdır. Buna göre biraz azaltılabilir.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
```

Sonuçlar:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
flatten (Flatten)	(None, 97200)	0
dense (Dense)	(None, 128)	12441728
dense_1 (Dense)	(None, 300)	38700
dense_2 (Dense)	(None, 2)	602
=====		

Total params: 12,481,030

Trainable params: 12,481,030

Non-trainable params: 0

Sonuçlar :

Epoch 5/10

330/330 [=====] - 16s 49ms/step - loss: 0.3651 - accuracy: 0.8413 - val_loss: 0.4731 - val_accuracy: 0.7824

Epoch 6/10

330/330 [=====] - 16s 49ms/step - loss: 0.3635 - accuracy: 0.8388 - val_loss: 0.3571 - val_accuracy: 0.8443

Epoch 7/10

330/330 [=====] - 16s 49ms/step - loss: 0.3447 - accuracy: 0.8506 - val_loss: 0.3356 - val_accuracy: 0.8591

Epoch 8/10

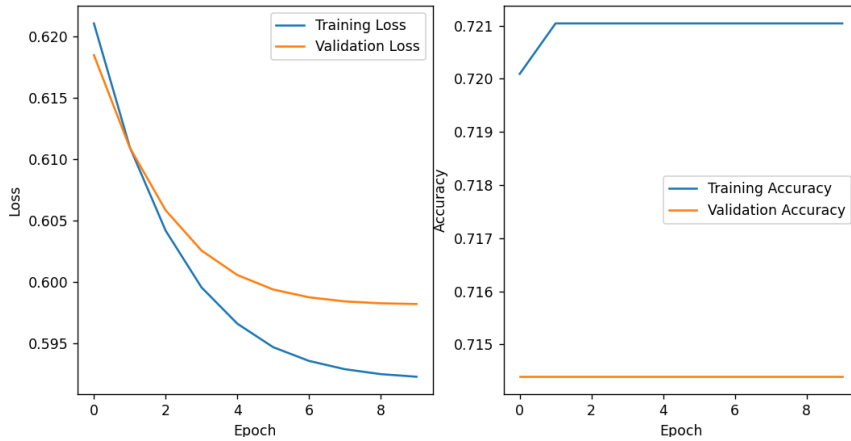
330/330 [=====] - 16s 48ms/step - loss: 0.3457 - accuracy: 0.8493 - val_loss: 0.3527 - val_accuracy: 0.8454

Epoch 9/10

330/330 [=====] - 16s 48ms/step - loss: 0.3483 - accuracy: 0.8516 - val_loss: 0.3735 - val_accuracy: 0.8299

Epoch 10/10

330/330 [=====] - 16s 48ms/step - loss: 0.3403 - accuracy: 0.8530 - val_loss: 0.3484 - val_accuracy: 0.8526



Grafik 10: Epok değerine göre kayıp ve doğruluk grafikleri

Öğrenme oranının değeri çok küçük olduğundan, model test veri setinde bir öncekilerden de daha kötü olmuştur.

3.3 Ara Katmanlardaki Nöronların Sayısını Azaltmak:

```
tf.keras.layers.Dense(150, activation='relu'),
```

Sonuçlar:

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 97200)	0
dense (Dense)	(None, 128)	12441728
dense_1 (Dense)	(None, 150)	19350
dense_2 (Dense)	(None, 2)	302

Total params: 12,461,380

Trainable params: 12,461,380

Non-trainable params: 0

Epoch 5/10

330/330 [=====] - 15s 46ms/step - loss: 0.3855 - accuracy: 0.8333 - val_loss: 0.5176 - val_accuracy: 0.7983

Epoch 6/10

330/330 [=====] - 16s 49ms/step - loss: 0.4259 - accuracy: 0.7929 - val_loss: 0.3768 - val_accuracy: 0.8530

Epoch 7/10

330/330 [=====] - 17s 52ms/step - loss: 0.3973 - accuracy:
0.8310 - val_loss: 0.3682 - val_accuracy: 0.8348

Epoch 8/10

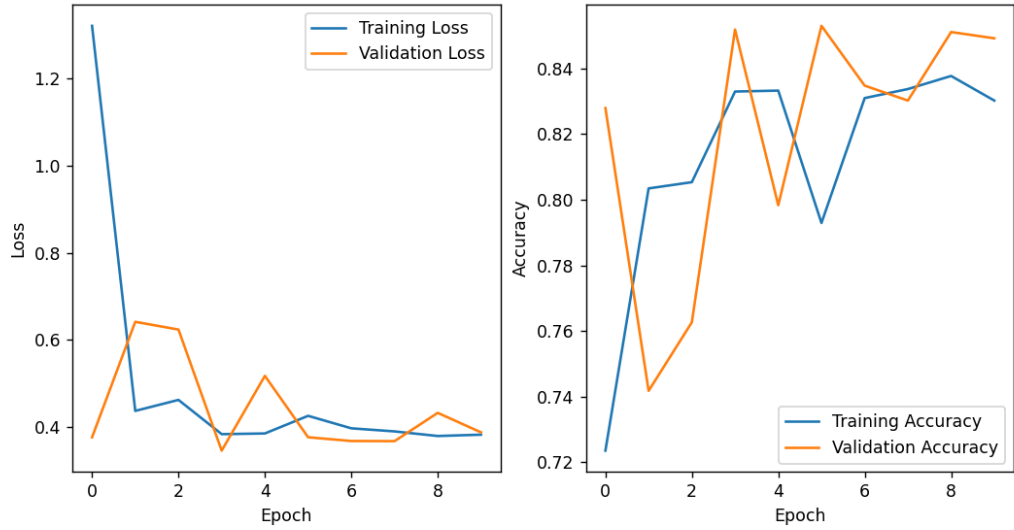
330/330 [=====] - 16s 49ms/step - loss: 0.3902 - accuracy:
0.8337 - val_loss: 0.3679 - val_accuracy: 0.8302

Epoch 9/10

330/330 [=====] - 17s 51ms/step - loss: 0.3797 - accuracy:
0.8377 - val_loss: 0.4327 - val_accuracy: 0.8511

Epoch 10/10

330/330 [=====] - 16s 49ms/step - loss: 0.3827 - accuracy:
0.8302 - val_loss: 0.3881 - val_accuracy: 0.8492



Grafik 4: Epok değerine göre kayıp ve doğruluk grafikleri

Modele yapılan bütün optimizasyonlara rağmen(her ne kadar bir öncekinden daha iyi olsa da) aynı düşük verime sahip olmaktadır.

3.4 Aktivasyon Fonksiyonun Değiştirilmesi:

Softmax'tan sigmoid'a geçiş yapıldığında aşağıdaki değişiklikler gözlemlenmektedir.

```
tf.keras.layers.Dense(2, activation='sigmoid')
```

Sonuçlar:

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
flatten (Flatten)	(None, 97200)	0
dense (Dense)	(None, 128)	12441728
dense_1 (Dense)	(None, 150)	19350
dense_2 (Dense)	(None, 2)	302
=====		

Total params: 12,461,380

Trainable params: 12,461,380

Non-trainable params: 0

Epoch 5/10

330/330 [=====] - 16s 48ms/step - loss: 0.3750 - accuracy: 0.8412 - val_loss: 0.3684 - val_accuracy: 0.8367

Epoch 6/10

330/330 [=====] - 16s 49ms/step - loss: 0.3860 - accuracy: 0.8158 - val_loss: 0.4567 - val_accuracy: 0.8112

Epoch 7/10

330/330 [=====] - 16s 48ms/step - loss: 0.4034 - accuracy: 0.8289 - val_loss: 0.3860 - val_accuracy: 0.8329

Epoch 8/10

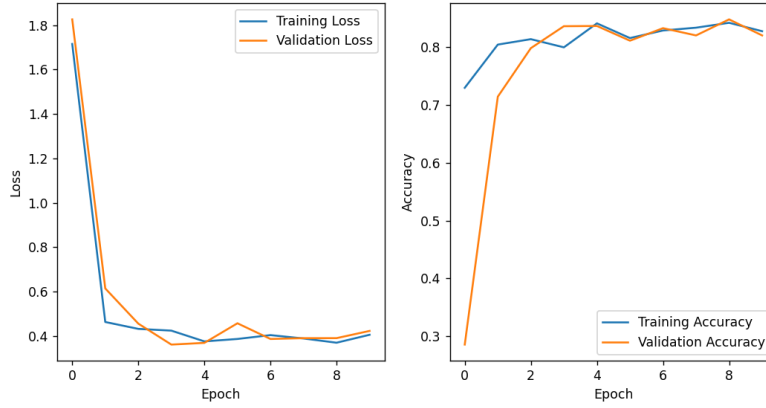
330/330 [=====] - 16s 48ms/step - loss: 0.3882 - accuracy: 0.8337 - val_loss: 0.3896 - val_accuracy: 0.8204

Epoch 9/10

330/330 [=====] - 16s 48ms/step - loss: 0.3691 - accuracy: 0.8422 - val_loss: 0.3897 - val_accuracy: 0.8481

Epoch 10/10

330/330 [=====] - 16s 48ms/step - loss: 0.4049 - accuracy:
0.8277 - val_loss: 0.4223 - val_accuracy: 0.8204



Grafik 11: Epok değerine göre kayıp ve doğruluk grafikleri

Çıkış katmanındaki aktivasyon fonksiyonunda önemli bir değişiklik yapılmıştır. Özellikle ikili sınıflandırma problemleri için oldukça kullanışlı olan sigmoid fonksiyon kullanılmıştır. Bu değişiklik sayesinde çok daha iyi sonuçlar elde edilmiştir.

Kodun colab bağlantısı:

<https://colab.research.google.com/drive/1xQVQV6ud-5eoQl0kp7o7nvzxLQ5ZZikM#scrollTo=rhqd-mz1AsCe>

Soru 6.

Soru 3 ve 5'te uyguladığınız iyileştirmeler ile elde ettiğiniz sonuçlara bakarak tüm modeller arasında hangi modelin ve hangi çeşit ağ mimarisinin en iyi performans gösterdiğini açıklayın? Nedenlerini açıklayın.

CNN'ler, yerel uzaysal bağıntıları ve özellikleri otomatik olarak öğrenme yetenekleri sayesinde özellikle görüntü işleme gibi alanlarda üstün performans gösterir. Ancak, daha karmaşık ve yüksek hesaplama gücü gerektirirler.

İleri Besleme Ağları (FFNN), daha basit ve geniş bir uygulama alanına sahip olup, daha hızlı ve kolay eğitilebilirler. Ancak, karmaşık veri yapılarını modellemek için yeterince güçlü değildir ve genellikle el ile özellik mühendisliği gerektirirler.

Yaptığımız uygulama için aynı şey geçerlidir. FFNN ne kadar basit olsa da giriş verilerimiz fotoğraflardan oluştuğu için az verim ile çalışmaktadır. Bunu da yapılan optimizasyonlardan anlaşılmaktadır.

